Software Requirements Specification

PROJECT: DEVSNAP — DEVELOPER'S SNIPPET DIARY

Author: D. Radha Kalyani **Target Timeline:** 48-Hour Sprint **Version:** 1.0 (Stable)

1. Introduction & Executive Intent

DevSnap is a web-based, highly curated micro-blogging utility engineered explicitly for software developers to document, publish, and archive architectural breakthrough snippets and complex debugging fixes.

Traditional social ecosystems treat technical statements poorly by collapsing nested formatting into unreadable string arrays, while visual screenshots degrade utility by locking text characters into flat pixel layouts. DevSnap targets this explicit problem space by rendering code assets as high-fidelity, interactive, AST-tokenized IDE frames natively within standard feed views. It serves directly as a public engineering ledger and an open "Today I Learned" (TIL) knowledge base.

2. System Architecture & Topology

To ensure rapid execution and continuous processing within a compressed weekend sprint matrix, the ecosystem completely bypasses complex custom server tiers in favor of a direct multi-channel **Serverless Headless Architecture**:

```
[ Client-Side Application Interface ] (Vite + React + Tailwind CSS)
            |
            ▼ (Secure Direct Target Ingestion / Supabase JS Client SDK)
[ Serverless Managed Cloud Engine ] (Supabase Core Infrastructure Stack)
            |
            ▼ (Real-Time Runtime Parsing Engine)
[ Visual Tokenization Layer ] (Prism Abstract Engine / react-syntax-highlighter)
```

3. Relational Data Dictionary Specification

The backend state engine utilizes a single relational database frame hosted natively inside a serverless PostgreSQL instance. The architecture requires execution of the following Data Definition Language (DDL) command block directly inside the target management workspace:

```sql
CREATE TABLE dev_snaps (
  id BIGSERIAL PRIMARY KEY,
  username VARCHAR(50) NOT NULL,
  title TEXT NOT NULL,
  code_content TEXT NOT NULL,
  language VARCHAR(30) NOT NULL DEFAULT 'javascript',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- High-speed index configuration for chronological array rendering
CREATE INDEX idx_dev_snaps_created_at ON dev_snaps (created_at DESC);

-- Open Access Row Level Security Profile for Fast-Cycle Prototyping
ALTER TABLE dev_snaps ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Allow global read and write access" ON dev_snaps FOR ALL USING (true) WITH CHE
```

4. Functional Requirements Matrix

Module ID	Feature Definition	Technical Implementation Blueprint
FR-01	Chronological Feed Hydration	React execution lifecycle interfaces handle dynamic array querying from the `dev_snaps` target index sorted descending. Data yields layout post blocks exposing user tags, text bodies, and token views.
FR-02	IDE Structural Mirroring	Client-side compilation of raw text payloads inside `react-syntax-highlighter` nodes utilizing abstract structural token definitions under a configured `vscDarkPlus` visualization package theme.
FR-03	Atomic Record Ingestion	A form capture modal registers standard string variables into localized state components. Submission actions fire standard client hooks passing parameter payloads straight into the Supabase database engine.
FR-04	Clipboard Stream Injection	An active interaction handler links specific trigger events straight to browser data transport pipelines via the standard `navigator.clipboard.writeText` runtime interface parameter string.

5. Non-Functional Requirements & Styling Language

- **UI Accent & Tone:** The visual profile enforces a strict dark interface layout. Background canvases must stay locked to deep neutral tones (`#0f172a` or slate black variants). High-visibility elements use electric yellow or cyan exclusively to mimic a developer environment.
- **Latency Targets:** Primary database calls bypass external layer processing to complete layout hydration cycles within 200 milliseconds of interface load tracking.
- **Layout Fluidity:** Layouts adapt natively using responsive Tailwind grid boundaries, wrapping data structures seamlessly for views matching mobile breakpoints up to widescreen configurations.

6. 48-Hour Execution Sprint Sequence

Hours 00 – 06: Environment Scaffolding
Initialize local project structures via Vite, integrate global Tailwind utilities, and configure client environmental connection keys.

Hours 06 – 18: Interface Layout Compositing
Build navigation elements and draft core rendering components wired with tokenized snippet highlights.

Hours 18 – 30: Database Data Binding
Link user interface capture elements to live database hooks, confirming write mechanics and network response tracking.

Hours 30 – 48: Optimization & Live Assembly
Deploy copy actions, refine layout padding, and release the live repository through high-speed cloud edge hosts like Vercel.

> **Vibe-Coding Workspace Tip:** Pass Section 3 and Section 4 blocks directly to your development tool workspace and state: *"Review this schema matrix and design standard React layouts for App.jsx and a database layer tracking client file connections to initiate Phase 1."*